

Harness Tri-Camada: memória como ambiente, API como tool, App como skill

Um runtime híbrido (ReAct + contexto-ambiente + skills) para o chat agêntico de marketing da AdzHub

Rafael Yran Azevedo

Candidato · Desafio Harness Agêntico · AdzHub Núcleo Fundacional, 2026

RESUMO. O gestor de marketing da AdzHub não precisa de um chatbot que responde: precisa de um agente que resolve tarefas compostas (cruzar gasto de anúncio com venda real, diagnosticar um CPA que subiu, montar a pauta de uma call) navegando o histórico da conta. Defendo que o erro comum, tratar tudo como *tool*, subutiliza o domínio. Proponho um harness híbrido de três mecanismos, um por camada: a memória (supercérebro: grafo + linha do tempo) é **hidratada como ambiente** antes do loop, não consultada às cegas; as APIs (Meta, CRM) são **tools** num loop ReAct; e os Apps de metodologia são **skills** encapsuladas que o harness invoca sem reimplementar. Um protótipo web de chat ilustra a tese sobre um dataset mock da Housewhey, com o *trace* do harness visível (hidratação → tools → observações → resposta) e cinco problemas de gestão plantados e não rotulados. O que fica só no paper: a construção do grafo temporal real, permissões de escrita e avaliação automática. O recorte consciente: nenhuma ação que escreve na conta; o MVP lê, diagnostica e propõe.

Palavras-chave: harness agêntico, ReAct híbrido, memória temporal, contexto como ambiente (RLM), skills, orquestração, marketing.

1. Introdução

O produto-alvo é um chat agêntico (no espírito do Cursor, mas para marketing): o gestor descreve uma tarefa e o agente orquestra contexto, Apps e APIs até uma ação útil. Antes de desenhar, separo dois conceitos que se confundem. O **modelo** é o LLM: prevê o próximo token e, no máximo, emite a intenção de chamar uma função. O **harness** é o runtime em volta: o loop que executa a chamada, injeta a observação de volta, guarda estado, impõe limites (allowlist de tools, teto de passos) e decide o que entra no contexto. Trocar de modelo troca o raciocínio; trocar de harness troca o que o agente consegue fazer com aquele raciocínio.

Uma imagem fixa a distinção. O LLM é um cérebro num pote: sabe muito, mas não vê o mundo nem age. O harness é o exoesqueleto que pluga esse cérebro na operação. O exoesqueleto restringe o foco (em vez de deixar o cérebro divagar, prende ele nos passos daquela tarefa), provê os músculos e as ferramentas (tools, memória, dados da conta) e força a saída num comportamento estrito. E faz isso *sem tocar no cérebro*: não mexemos nos pesos do modelo (isso seria treino), apenas embrulhamos software em volta e filtramos o que o LLM vê. Mesmo cérebro, exoesqueletos diferentes, agentes diferentes.

No início do estudo eu tratava harness como sinônimo de "loop de tool-calling": ligar o modelo a um punhado de funções e deixar rodar. Estudando os tipos de referência (ReAct [2], sandbox/CodeAct [3], sessão com permissões e skills, orquestração por grafo, contexto como ambiente [4]) e o SDK do OpenHands [1], o que mudou foi entender que a decisão de arquitetura não é "qual loop", e sim **como cada recurso do domínio entra no runtime**: o que vira tool, o que vira

memória e o que vira unidade de trabalho encapsulada. Um bom harness é, antes de tudo, uma política de contexto.

Tese. Para este domínio, um harness que trata suas três camadas com o mesmo mecanismo (tudo tool) é caro e frágil; defendo um híbrido em que *memória é ambiente, API é tool e App é skill*. As seções §2 a §6 definem o vocabulário, justificam a escolha contra as alternativas, mostram o que o protótipo ilustra e onde a solução quebra.

2. Preliminares

2.1. O que é um harness (no meu vocabulário)

- **loop**: o ciclo intenção → ação → observação → intenção, repetido até uma resposta final ou um limite.
- **tool**: função com assinatura declarada que o modelo pode pedir para chamar; devolve uma observação (JSON) que volta ao contexto.
- **observação**: o resultado de uma tool, reinjetado como nova evidência.
- **memória**: estado que o harness carrega e injeta sem o modelo precisar "lembrar" de buscá-lo.
- **allowlist / max_steps**: os guard-rails que separam um harness de um chatbot solto: o conjunto fechado de tools chamáveis e o teto de passos por turno.
- **estado**: o que sobrevive entre passos (mensagens, resultados, escolhas) e, num produto, entre turnos.

2.2. O domínio AdzHub (como eu o leio)

A página do desafio expõe três camadas. (i) **Supercérebro**: a memória da operação, pessoas, cliente, campanhas e decisões ligadas em grafo (GraphRAG), com linha do tempo (contexto temporal). (ii) **Apps**: metodologias empacotadas (insight da semana, brief de criativo, diagnóstico, análise de criativos,

mapa de solução). (iii) **APIs**: os dados da operação (Meta, Google, Analytics, CRM, WhatsApp). A observação central deste paper: essas três camadas têm *naturezas* diferentes, e o harness rende mais quando respeita essa diferença em vez de achatar tudo em "função que o LLM chama".

3. Arquitetura

3.1. A escolha: um híbrido de três mecanismos

Escolho um híbrido próprio com núcleo ReAct, e justifico contra cada referência, não só descrevo a minha:

- **ReAct puro** (loop tool-calling): é o esqueleto certo para tarefas abertas de diagnóstico, mas, sozinho, transforma memória em "mais uma tool", e o LLM precisa adivinhar quando buscar contexto. Num domínio onde quase toda pergunta pressupõe o histórico da conta, isso gera respostas sem chão ou passos desperdiçados. Mantenho o loop, mas tiro a memória de dentro dele por padrão.
- **Sandbox / CodeAct** (o agente escreve e executa código [3]): poderoso para engenharia, mas aqui o trabalho não é rodar código arbitrário, é cruzar fontes tabulares e aplicar metodologia. O sandbox soma superfície de risco

(execução) e latência sem resolver o problema do gestor. Fica fora do MVP; volta, no máximo, como uma tool isolada de cálculo (§6).

- **Sessão com permissões e skills**: a parte de *skills* eu adoto (os Apps são exatamente isso). A parte de *permissões* eu adio de propósito, porque o MVP é só-leitura (§3.3). Uso metade dessa referência, conscientemente.
- **Orquestração por grafo** (estados): ótima quando o fluxo é conhecido e repetível (um pipeline fixo). O chat do gestor é o oposto: aberto, uma tarefa diferente por mensagem. Um grafo de controle rígido engessaria. Nota: o supercérebro é um grafo de *dados*, não de *controle*. Uso o grafo como memória, não como máquina de estados.
- **Contexto como ambiente (RLM [4])**: é a peça que cura a fraqueza do ReAct puro. Trato o supercérebro como ambiente: o harness recupera um pacote de contexto (nós relevantes do grafo + eventos recentes da timeline) e o injeta no prompt antes do loop. O modelo já começa situado.

O híbrido, então: **RLM para a memória, ReAct para as APIs, skills para os Apps**. Nenhuma referência sozinha cobre as três naturezas; a tese está na composição.

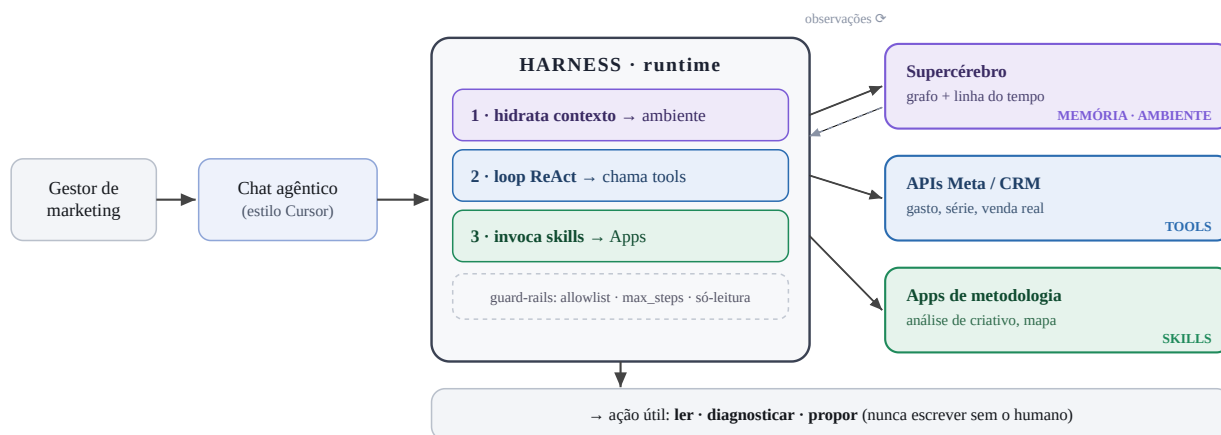


FIGURA 1. O harness Tri-Camada. O gestor fala com o chat; o harness (1) hidrata a **MEMÓRIA** do supercérebro como ambiente antes do loop, (2) roda o loop ReAct chamando as **APIs** como tools e (3) invoca os **Apps** como skills de metodologia; guard-rails (allowlist, max_steps, só-leitura) cercam o loop. O que é *tool*: dado cru da operação. O que é *memória*: contexto temporal, injetado. O que é *app*: metodologia encapsulada, invocada.

3.2. O que é tool, o que é memória, o que é app

Memória (supercérebro), hidratada como ambiente. Antes do loop, o harness chama uma recuperação (`search_client_context` + `get_timeline`) semeada pela intenção e injeta um pacote compacto no *system prompt*. Durante o loop, o modelo ainda pode aprofundar (buscar uma ata com `search_conversations`). Memória é ambiente por padrão, tool sob demanda.

API (Meta, CRM), tools clássicas. Sem estado, chamadas dentro do loop: `list_ads` e `get_ad_insights` (gasto, série semanal, saturação), `get_leads` (venda real). O join Meta×CRM por `utm_content = ad_id` é trabalho do *agente*, não da tool: é aí que mora a inteligência de "não confiar só no gerenciador".

App (metodologia), skills encapsuladas. `run_app_analise_criativos` devolve ranking + recomendação (seguir/pausar/variá); `get_mapa_solucão` devolve a ficha da marca (o que não pode falar). O harness chama o app e usa a saída; não recria a metodologia no prompt. Isso mantém a metodologia versionável no produto e barata em tokens.

3.3. Trade-offs aceitos de propósito

- **Fidelidade vs simplicidade**: a hidratação é uma recuperação simples (match no grafo + últimos N eventos), não um recuperador semântico com embeddings. Aceito recall menor por latência baixa e previsibilidade; num MVP, contexto errado é pior que contexto raso.
- **Latência vs profundidade**: `max_steps = 8` e allowlist fechada. Prefiro cortar cedo e responder com o que há a

- divagar.
- **Custo:** Apps como skills e `get_leads` com sumário cortam tokens; a metodologia não é reprocessada pelo LLM a cada turno.
 - **Segurança:** só-leitura. Nenhuma tool escreve na conta (não pausa anúncio, não mexe em verba). O agente propõe; o humano executa. Remove a classe inteira de risco de ação indevida e é coerente com "o gestor decide".
 - **Simplicidade do MVP:** um cliente, tools que leem mocks. Sem multi-tenant, sem auth, sem escrita.

4. Sistema

4.1. O que o protótipo ilustra

Um chat web (estilo Cursor) roda o harness sobre um dataset mock da Housewhey. Cada turno mostra, num painel de *trace*, as fases do harness: hidratação do supercérebro (fase), depois as tool calls (com *badge* por camada: memória/api/app), com args e observação inspecionáveis, e a resposta ancorada nos números. Dois motores compartilham o mesmo trace: **(a) Simulado**, sem chave, um planejador roteirizado que executa as tools reais e compõe a resposta a partir do dado (prova que o dado sustenta a conclusão); **(b) LLM**, via OpenRouter, o loop ReAct de verdade decide as chamadas.

Tabela 1. Peças do harness: o que é cada uma e onde vive.

Peça (tool)	Camada	Papel na tese	Estado no protótipo
<code>search_client_context</code>	Supercérebro · grafo	Memória (ambiente): quem/o quê da conta	simulado sobre mock; visível no trace
<code>get_timeline</code>	Supercérebro · temporal	Memória (ambiente): histórico recente	simulado sobre mock; visível no trace
<code>search_conversations</code>	Memória de canal	Memória (sob demanda): o porquê (aprovação)	simulado sobre mock; visível no trace
<code>list_ads</code> / <code>get_ad_insights</code>	API · Meta Ads	Tool: gasto, série, saturação	simulado sobre mock; visível no trace
<code>get_leads</code>	API · CRM	Tool: venda real, atribuição	simulado sobre mock; visível no trace
<code>run_app_analise_criativos</code>	App · metodologia	Skill: ranking + recomendação	simulado sobre mock; visível no trace
<code>get_mapa_solucão</code>	App · marca	Skill: restrições ("não pode falar")	simulado sobre mock; visível no trace

4.3. OpenRouter e troca de modelo

No painel `Configurar`, o avaliador escolhe o motor (Simulado ou LLM), cola a `OPENROUTER_API_KEY` (guardada só em `sessionStorage`, nunca num servidor) e digita o id do modelo (dropdown com sugestões que suportam tool-calling; campo livre). Trocar de modelo é trocar o texto do campo. O modo Simulado dispensa chave e serve o avaliador que só quer ver o harness orquestrar.

5. Notas de execução

Um turno concreto (intent → memória/tools → observação → resposta), para "o CPA do Ômega 3 subiu, diagnostique":

- **Hidratação:** nós do grafo (campanha Ômega 3, time, tarefa de aprovação) + timeline recente entram no prompt.
- **Loop:** `list_ads` (gasto por anúncio) → `get_leads` (venda real) → `get_ad_insights` (hook_rate 32% → 18%, frequência 1,8 → 4,8) → `run_app_analise_criativos` (recomenda pausar).
- **Observações combinadas:** o criativo de depoimento tem CPA R\$ 2.100 contra R\$ 100 do antes/depois, saturou e sozinho estourou o orçamento (R\$ 6.200 vs R\$ 5.000).
- **Resposta:** causa raiz + próximo passo: pausar o depoimento resolve CPA, fadiga e verba de uma vez.

4.2. Datasets: o que é fake, o que dá para testar

Sete arquivos JSON (um cliente, cruzados: `ad_id` = `utm_content`, pessoas no grafo, timeline e WhatsApp). Cada arquivo é uma tool (Tabela 1). Tudo é fake, mas coerente ponta a ponta; **cinco problemas de gestão estão plantados e não rotulados**: CPA estourado, criativo saturado, origem declarada que mente contra o UTM, aprovação travada por *claim* proibido, e spend vs budget. O avaliador cola três prompts (relatório, diagnóstico, origem dos leads) e vê o harness orquestrar duas ou mais tools e chegar à conclusão sem que ela esteja escrita no dado.

PDF anexo (este). Demo opcional: chat web estático, roda de `file://` por duplo clique ou de qualquer host, sem backend. LLM via OpenRouter, chave só na sessão do browser. Dados mock gerados de forma determinística a partir do `dataset_prompt`. O que é fake: todas as métricas e nomes. O que "parece real": o cruzamento e o trace, porque as tools leem o mesmo dado que a resposta cita, e o avaliador pode abrir cada passo.

6. Limitações

Onde a solução quebra, por tarefa do gestor:

- **Relatório:** o join depende de `utm_content` limpo. No real, UTM faltando ou errado, múltiplos toques (last-click vs multi-touch) e janelas de atribuição divergentes entre Meta e CRM quebram o "custo por venda" exato. O mock tem UTM limpo; a conta real, não.
- **Diagnóstico:** a hidratação rasa (sem embeddings) pode não trazer o nó certo se a pergunta usa um sinônimo que não casa por substring. E o agente *correlaciona*, não prova causa: "o depoimento saturou" é leitura plausível, não teste estatístico.
- **Pauta:** depende de timeline e conversas populadas e datadas; com memória esparsa, a pauta fica genérica. O harness não sabe o que não foi registrado.
- **Criativos:** o App recomenda pausar/variá, mas o *brief* novo é gerado pelo LLM; sem o mapa de solução no contexto, ele pode propor copy que a marca não pode falar. Mitigo injetando `get_mapa_solucão`, mas não garanto.

Além disso: só-leitura por escolha (não fecho o loop de execução); um cliente só (sem multi-tenant nem auth); memória mock, o grafo temporal de verdade (resolução de entidade, invalidação, decaimento) é o trabalho pesado que fica de fora.

Com mais uma semana eu construiria: (a) recuperação de memória de verdade, um grafo temporal com embeddings e

decaimento (no espírito de Graphiti/Zep [5]), para a hidratação parar de depender de match por substring; (b) uma tool isolada de verificação numérica, o único lugar onde um mini-sandbox se paga, para o join Meta×CRM ficar auditável; (c) confirmação de escrita (pausar anúncio) com o humano no meio. **O que eu deliberadamente não construiria:** um grafo de controle por cima do chat (mata a abertura da tarefa), execução de código arbitrário no MVP (risco sem retorno) e memória de longo prazo que aprende sozinha sem revisão, porque num domínio com o dinheiro do cliente, contexto errado propaga erro.

Referências

1. OpenHands. *The OpenHands Software Agent SDK*. arXiv:2511.03690, 2026.
2. S. Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629, 2023.
3. X. Wang et al. *Executable Code Actions Elicit Better LLM Agents (CodeAct)*. arXiv:2402.01030, 2024.
4. *Recursive Language Models*. arXiv:2512.24601, 2026.
5. P. Rasmussen et al. *Zep / Graphiti: memória de agente por grafo de conhecimento temporal*. 2025.
6. Anthropic. *Building Effective Agents*. Nota de engenharia, 2024.

Documento do candidato para o Desafio Harness Agêntico da AdzHub.
Nomes de cliente, tools e métricas são ficção ilustrativa (mock Housewhey).